

Slay The Lord

P박대원, I임찬진, P박경준

한국공학대학교 게임공학과

june25656@tukorea.ac.kr ckswlsdl0916@gmail.com pkj7323@tukorea.ac.kr

DirectX 12 기반 독자적 glTF 파서 및 스키닝 애니메이션 파이프라인 설계 및 최적화

요 약

본 연구는 3D 그래픽스 표준 포맷인 glTF의 공식 명세를 심층 분석하여, Assimp와 같은 외부 라이브러리에 의존하지 않는 자체적인 파서 및 렌더링 파이프라인을 DirectX 12 환경에 구축한 과정을 다룬다. 핵심 모듈인 ReadGLTFMesh를 중심으로 메쉬의 정점 데이터를 추출, 이기종 좌표계 변환, 그리고 뼈대 행렬 보간을 구현하였다. 또한, 이진 탐색 알고리즘을 통한 애니메이션 키프레임 탐색 최적화, 다중 프레임 렌더링 환경을 고려한 LinearAllocator 기반의 메모리 할당 구조, 그리고 복합 메쉬를 위한 마스터 스켈레톤(Master Skeleton) 구조 및 SocketComponent를 설계하였다. 또한 무게중심 좌표계 기반의 파티클 추출 로직을 설계하였고, 계층적 OBB를 활용한 BVH 구조를 통해 카메라와 건물 간의 충돌 연산 효율을 높이는 기술들을 개발하여, 게임 엔진의 성능과 유연성을 향상시켰다.

1. 서 론

1.1 연구 배경 및 목적

glTF는 계층적 노드 구조, PBR 재질, 스키닝 애니메이션 등 방대한 데이터를 효율적으로 담아내는 표준 포맷으로 활용되고 있다. 하지만 이를 게임 엔진에 통합하기 위해 범용 외부 라이브러리(Assimp 등)를 사용할 경우, 프로젝트에 불필요한 오버헤드가 발생하고 파이프라인의 세밀한 제어가 어려워진다. 본 연구는 glTF 표준 명세(Specification)를 직접 분석하고 바이너리 데이터를 직접 파싱하는 시스템을 구축하여, DirectX 12 기반의 자체 게임 엔진에 다양한 기능들을 제공하고, 최적화된 렌더링 환경을 구축하는 것을 목적으로 한다.

2. 개발 내용

2.1 glTF 표준 명세 분석 및

ReadGLTFMesh 파서 구현

파이프라인의 핵심인 ReadGLTFMesh 클래스는 JSON 파일의 구조와 uri로 연결된 바이너리 버퍼(.bin)를 직접 읽어 들인다.

정점 및 스키닝 데이터 로드: glTF의 accessors와 bufferViews 명세에 따라 정점 좌표, 노멀, UV, 탄젠트뿐만 아니라 스키닝에 필수적인 조인트 인덱스(JOINTS_0)와 가중치(WEIGHTS_0) 데이터를 SCALAR, VEC3, VEC4 타입에 맞게 바이트 스트라이드(Byte Stride)를 계산하여 추출하였다.

이기종 좌표계 변환: glTF의 오른손 좌표계(Right-Handed)와 열 우선(Column-Major) 행렬 포맷을 DirectX 12의 왼손 좌표계(Left-Handed)와 행 우선(Row-Major) 환경에 맞추기 위해,

Z축을 반전시키는 스케일 행렬 $S(1, 1, -1)$ 을 역바인드 행렬(Inverse Bind Matrix) 변환식 양단에 곱해 모든 회전과 이동 성분이 정확히 변환되도록 수학적 처리를 거쳤다.

2.2 뼈대 행렬 보간 및 키프레임 탐색 알고리즘 최적화

AnimationComponent의 제어를 받는 애니메이션 시스템은 정밀한 보간(Interpolation) 연산을 요구한다.

다양한 보간 방식 지원: glTF 명세에서 제공하는 Linear, Step, Cubic Spline 보간 방식을 완벽히 분석하여, 시간 변화율(t)에 따라 Translation(이동), Rotation(회전, Quaternion Slerp), Scale(크기) 값을 프레임마다 연산하도록 구현하였다.

탐색 알고리즘 최적화: 매 프레임 현재 애니메이션 재생 시간에 맞는 키프레임을 찾기 위해 선형 탐색(Linear Search)을 사용할 경우 $O(N)$ 의 비용이 발생한다. 이를 최적화하기 위해 시간 배열이 정렬되어 있다는 특성을 활용, std::upper_bound 기반의 이진 탐색(Binary Search)을 도입하여 탐색 시간 복잡도를 $O(\log N)$ 으로 단축시켰다.

2.3 마스터 스켈레톤(Master Skeleton) 구조 및 소켓(Socket) 시스템

투구, 갑옷 등 여러 파츠로 나뉜 캐릭터를 단일 개체처럼 렌더링하기 위해 마스터 스켈레톤 구조를 설계하였다.

뼈대 매핑 구조: 0번 스킨(Skin)을 메인 몸통(Master Skeleton)으로 삼고, 추가되는 파츠 메쉬들의 조인트 인덱스가 메인 몸통의 뼈대 인덱스를 참조하도록 local_to_master_map을 구성하여 렌더링 일관성을 확보하였다.

SocketComponent 구현: 특정 뼈대(예: 오른손)에 무기와 같은 별도의 GameObject를 부착하기 위한 시스템을 개발하였다.

장착된 객체의 최종 월드 행렬 $M_{\{final\}}$ 은 다음과 같은 행렬 곱셈을 통해 도출되어 캐릭터의 애니메이션과 완벽히 동기화된다.

$$M_{final} = M_{local} \times M_{socket} \times M_{world}$$

2.4 다중 프레임 렌더링을 위한 LinearAllocator 기반 메모리 최적화

다수의 애니메이션 객체가 렌더링 될 때, 뼈대 행렬을 갱신하기 위해 매번 상수 버퍼 리소스를 생성 및 해제하면 극심한 오버헤드가 발생한다. 다중 프레임 버퍼링(Multi-Frame Buffering) 환경을 지원하기 위해 LinearAllocator를 직접 설계하였다.

전체 프레임 수(Swap Chain Buffers)를 수용할 수 있는 거대한 업로드 힙(Upload Heap)을 초기 1회만 할당하였다.

DirectX 12 상수 버퍼의 256바이트 정렬(Alignment) 규칙(CB_ALIGNMENT)을 엄격히 적용하여 프레임별 메모리 블록을 분할하였다.

매 프레임 AnimationComponent가 뼈대 행렬을 복사할 때, 할당기의 오프셋만 순차적으로 증가시키며 $O(1)$ 의 비용으로 즉시 메모리를 빌려오는 구조를 채택하여 메모리 단편화와 동적 할당 부하를 원천 차단하였다.

2.5 계층적 OBB 및 BVH 구조를 활용한 카메라 충돌 연산 최적화

건물과 같은 거대 메쉬와 카메라 간의 실시간 충돌 처리를 위해 계층적 OBB(Oriented Bounding Box) 연산 구조를 설계하였다.

BVH 설계: 모든 세부 프리미티브(Primitive) 단위로 개별 바운딩 박스를 생성하고, 이를 모두 포괄하는 상위

수준의 '마스터 OBB'를 구축하여 일종의 BVH(Bounding Volume Hierarchy) 역할을 수행하도록 하였다.

충돌 오버헤드 감소: 카메라 충돌 검사 시 마스터 OBB와의 교차 여부를 먼저 판별하고, 충돌이 발생한 경우에만 하위 프리미티브의 OBB들을 순차적으로 검사하는 이단계 구조를 채택하였다. 이를 통해 건물과 같이 정점 데이터가 방대한 객체와의 교차 연산 오버헤드를 대폭 감소시켰다.

2.6 무게중심 좌표계를 활용한 표면 파티클 추출

메쉬 표면에서 균일한 파티클을 생성하기 위해 삼각형 면적 기반의 가중치 난수 알고리즘을 도입하였다. 무게중심 좌표계(Barycentric Coordinates)를 적용하여 정점 내부에 뭉침 없는 균일한 타겟 좌표를 생성하는 로직을 구현하여 파티클 이펙트가 타겟 메쉬의 형태를 서서히 갖춰가도록 처리하였다.

3. 결 론

3.1 개발 요약

본 연구는 Assimp와 같은 범용 외부 파싱 라이브러리에 의존하지 않고, DirectX 12 환경에 최적화된 독자적인 glTF 파서 및 스키닝 애니메이션 파이프라인을 구축하였다. JSON 명세와 바이너리 버퍼를 직접 분석하여 정점 및 가중치 데이터를 추출하였고, 수학적 연산을 통한 이기종 좌표계 변환을 완벽히 수행하였다. 또한, 이진 탐색 $O(\log N)$ 기반의 키프레임 보간 최적화, 다중 프레임 환경을 위한 선형 할당기(Linear Allocator) 기반의 메모리 최적화, 계층적 OBB 기반 BVH 연산, 장비 장착을 위한 마스터 스켈레톤 및 소켓 시스템을 구현하여 엔진의 렌더링 성능과 확장성을 크게 향상시켰다.

3.2 장점

외부 종속성 제거 및 경량화: 무거운 범용 라이브러리를 배제하고 필수적인 스키닝 및 메쉬 데이터만 직접 파싱하여 파이프라인의 무게를 줄이고 원하는 기능을 추가할 수 있는 세밀한 제어권을 확보하였다.

CPU 연산 및 메모리 할당 최적화: 애니메이션 보간 시 이진 탐색을 도입하여 키프레임 탐색 비용을 최소화하였고, BVH 구조의 OBB 연산을 통해 카메라 충돌 오버헤드를 최소화하였으며, LinearAllocator를 통해 상수 버퍼 갱신 시 발생하는 동적 메모리 할당 오버헤드와 단편화를 원천적으로 차단하였다.

유연한 객체 확장성: 마스터 스켈레톤 구조를 통해 분리된 캐릭터 파츠들을 하나의 뼈대로 통합 제어할 수 있으며, 소켓(Socket) 컴포넌트의 행렬 연산을 통해 애니메이션과 완벽히 동기화되는 장비 장착 시스템을 구현하였으며, 파티클 이펙트를 위한 메쉬의 표면 파티클 추출과 같이 엔진에 필요한 기능들을 유연하게 확장하였다.

3.3 향후 개발 방향

GPU 기반 컴퓨트 셰이더(Compute Shader) 스키닝 도입: CPU에서 수행 중인 뼈대 변환 행렬 연산을 컴퓨트 셰이더로 오프로딩(Off-loading)하여 GPU의 강력한 병렬 처리 능력을 활용, 성능 한계를 극복할 계획이다.

애니메이션 블렌딩 구축: 캐릭터 동작 전환 시의 부드러운 연결을 위해 가중치 기반의 애니메이션 블렌딩 알고리즘을 고도화할 예정이다.

멀티 스레드 기반 비동기 로딩 및 연산 분산: 메인 스레드의 부하를 줄이기 위해 멀티 스레딩 환경을 도입하여, 비동기적인 glTF 리소스 로딩과 애니메이션 계층 구조 갱신을 병렬로 처리하도록 구조를 고도화할 것이다.

문제 해결을 위한 연락처

june25656@tukorea.ac.kr
ckswlsdl0916@gmail.com
pkj7323@tukorea.ac.kr